

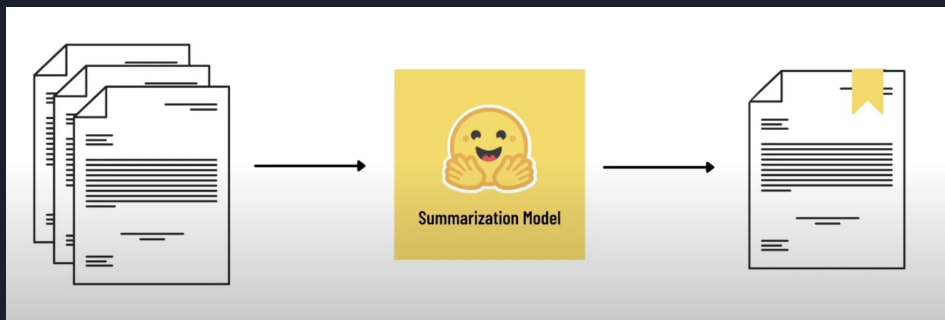


Summarization

Ismail, Pratyay, and Richard

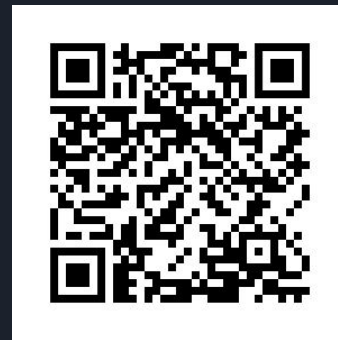
Introduction

- The task of text summarization uses Transformer models to condense long documents into summaries.
- Summarization tries to understand long passages and capture the main topics in the document.
- This tutorial investigates bilingual summarization for user-generated reviews.



Dataset

- Multilingual Amazon Reviews Corpus
 - Consists of Amazon product reviews in six languages.
 - Each review contains a body to be summarized, a title to be used as the target, and a language tag.
 - The data no longer exists at the location in the tutorial, it now exists here:
https://huggingface.co/datasets/dnagpt/kaggle_amazon_reviews_multi



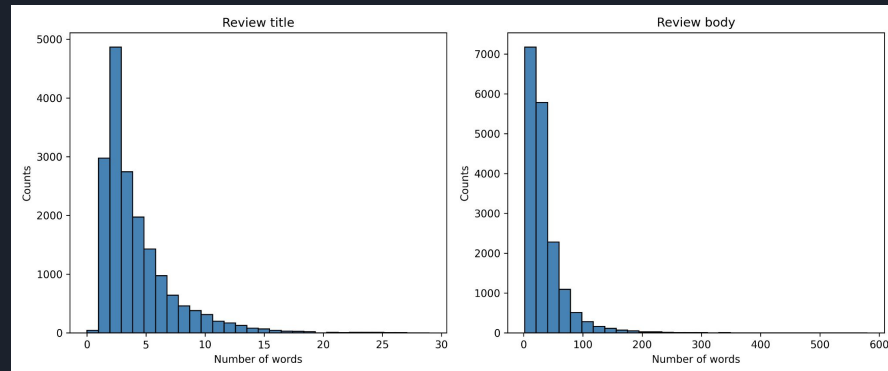
The python code for downloading the data sets as parquet files is available through the QR code.

Title Filtering

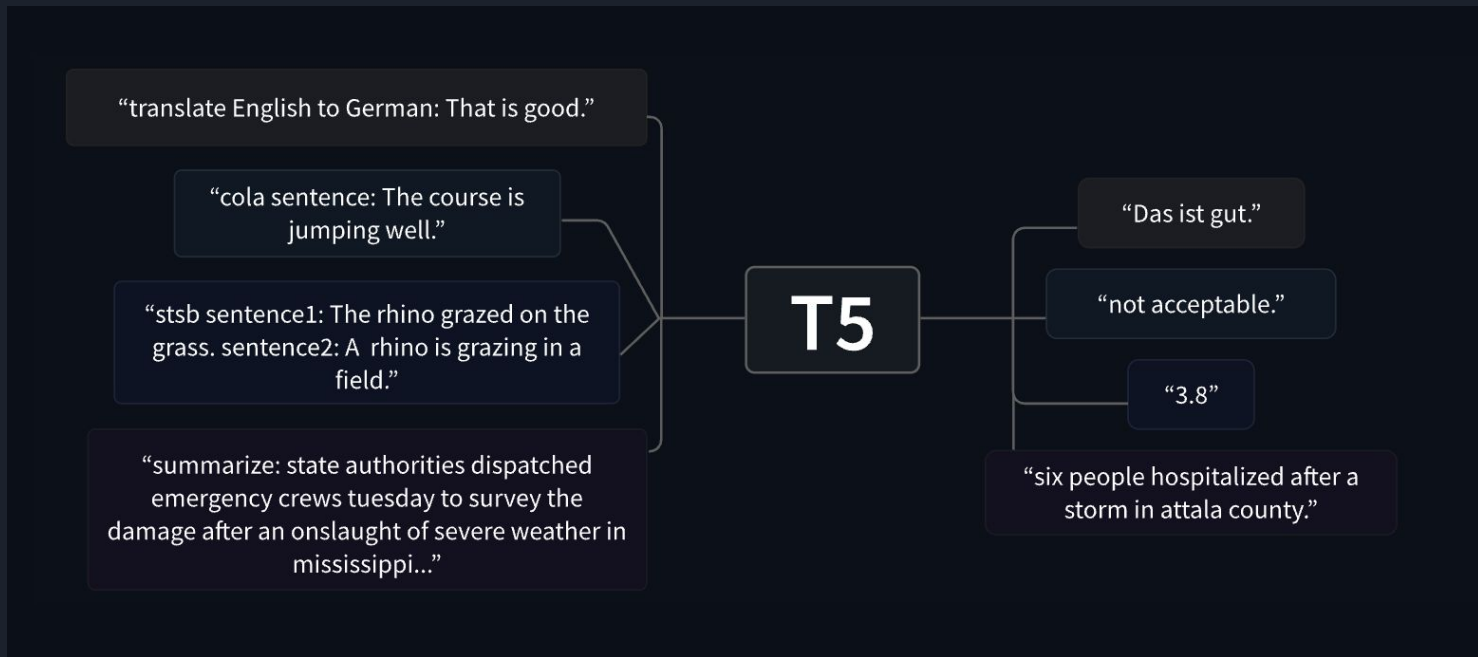
In summarization tasks, it is very important to check the distribution of words in the reviews and their titles. Because short reference summaries can bias the model to only output one or two words in the generated summaries.

So we'll take our shuffled and concatenated data and create some histograms to check how skewed the data is to short titles and short bodies of text.

Better if we remove the reviews with titles not containing at least 3 words.



MT5 Model





The transformer model

mt5-small: pretrained on the multilingual common crawl corpus mC4, covering 101 languages, trained in text-to-text framework.

This model is going to take in the review body as the input text and the title as the target text, or the summary that we want to create for the body.

The first step is to tokenize and encode the reviews and the titles.

This model uses a Sentence piece tokenizer, which DOES NOT split text at spaces, it works with raw text directly and encode everything as subword units, for example a space will be encoded as a underscore.

In this model we will also need to truncate the text because the mt5-small has a 512 token maximum input length.

encoder-decoder transformer model:

- encoder: reads the input text and creates a numerical understanding of it
- decoder: uses that understanding to generate the output text



The ROUGE score

REcall-Oriented Understudy for Gisting Evaluation

Precision →

How many of the words in the generated summary also appear in the reference summary?

$$\text{Precision} = \frac{\text{overlap words}}{\text{words in generated summary}}$$

Recall →

How many of the words in the reference summary were captured by the generated summary?

$$\text{Recall} = \frac{\text{overlap words}}{\text{words in reference summary}}$$

F1 (F-measure) →

A balance between precision and recall

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$



Rouge score and F1 score



Rouge score

ROUGE-1	Word overlap	Any task	Counts matching words.
ROUGE-2	Phrase overlap	Any task	Counts matching 2-word pairs.
ROUGE-L	Longest sequence overlap	Short text	Looks at word order and flow.
ROUGE-LSum (ROUGE-LM)	Longest sequence overlap across sentences	Summarization	Evaluates multi-sentence summaries (paragraph-level).



lead-3 Baseline

- We'll want a baseline for our model to see that it is doing well against a non-smart model.
- We'll use the sentence tokenizer from the nltk package and extract the first three sentences in a review. NLTK will split the first three sentences into each sentence onto one new line.
- Then we compare the first three sentences from the review and compare it with the title using the rouge score.
- We will use this rouge score and compare it with the one from the model to see that our model is performing better than if we were to just use the first three sentences of the review body as the summary.



Data Collator

- also you need to set up a data collator to shift the labels one position to the right.
- So [the, book, was, great, </s>] becomes [<pad>, the, book, was, great] the causes the the decoder to predict the next token only based on the ones before it. It is basically creating a padding that doesn't allow the decoder to peak ahead.
- With all of the above steps (list steps) you are ready to run your training function.



Fine tuning the model with accelerator

everything we have talked about to know how has to do with the Seq2SeqTrainer, if you want to be a little more deliberate with your training you can also use Accelerator and build the training model with more custom specifications.

for this more manual method you need to:

- create a DataLoader -> set the datasets to torch because the PyTorch dataloaders expect batches of tensors.
- with the datasets in tensors the next thing to do is instantiate the Data Collator and provide that with a fresh version of the model
- Define the optimizer, the tutorial says to use AdamW
- Then feed the model, optimizer, and dataloaders to the accelerator.prepare method